Exercise 1: Employees and Departments

- Every employee works in some department
- Every department has a manager
- If an employee works in a department and someone manages that department, the employee reports to that person

Goal: Every employee reports to a manager

1. The theory

Goal G: $\forall x [employee(x) \rightarrow \exists y (manager(y) \wedge reports_to(x, y))]$

#	Axiom
A1	$\forall x [employee(x) \rightarrow \exists y (department(y) \wedge works_in(x, y))]$
A2	$\forall x [department(x) \rightarrow \exists y (manager(y) \wedge manages(y, x))]$
A3	$\forall x \forall y \forall z [(works_in(x, y) \wedge manages(z, y)) \rightarrow reports_to(x, z)]$

Ex. 1: Conversion to clauses (1/2)

A1

Eliminate →: ∀x [¬employee(x) ∨ ∃y (department(y) ∧ works_in(x, y))]
Skolemize: y is in the scope of ∀x, so replace it with a function f(x):
∀x [¬employee(x) ∨ (department(f(x)) ∧ works_in(x, f(x)))]
Distribute ∨ over ∧ and drop ∀:
C1: ¬employee(x) ∨ department(f(x))
C2: ¬employee(x) ∨ works_in(x, f(x))

A2

Eliminate →: ∀x [¬department(x) ∨ ∃y (manager(y) ∧ manages(y, x))]
Skolemize with g(x):
∀x [¬department(x) ∨ (manager(g(x)) ∧ manages(g(x), x))]
Distribute ∨ over ∧ and drop ∀:
C3: ¬department(x) ∨ manager(g(x))
C4: ¬department(x) ∨ manages(g(x), x)

Ex. 1: Conversion to clauses (2/2)

A3

Eliminate →: ∀x ∀y ∀z [¬(works_in(x, y) ∧ manages(z, y)) ∨ reports_to(x, z)]

De Morgan:

C5: ¬works_in(x, y) ∨ ¬manages(z, y) ∨ reports_to(x, z)

¬G

¬∀x [employee(x) → ∃y (manager(y) ∧ reports_to(x, y))]

Eliminate →: ¬∀x [¬employee(x) ∨ ∃y (manager(y) ∧ reports_to(x, y))]

Push ¬ inward: ∃x [employee(x) ∧ ∀y ¬(manager(y) ∧ reports_to(x, y))]

De Morgan: ∃x [employee(x) ∧ ∀y (¬manager(y) ∨ ¬reports_to(x, y))]

Skolemize: the ∃x is outermost, so it becomes a constant a:

employee(a) ∧ ∀y (¬manager(y) ∨ ¬reports_to(a, y))

Split:

C6: employee(a)

C7: ¬manager(y) ∨ ¬reports_to(a, y)

Ex. 1: Clause set

Clause set

C1: ¬employee(x) ∨ department(f(x))
C2: ¬employee(x) ∨ works_in(x, f(x))
C3: ¬department(x) ∨ manager(g(x))
C4: ¬department(x) ∨ manages(g(x), x)
C5: ¬works_in(x, y) ∨ ¬manages(z, y) ∨ reports_to(x, z)
C6: employee(a)
C7: ¬manager(y) ∨ ¬reports_to(a, y)

Ex. 1: Resolution

Resolution

C8: department(f(a)) from C6 and C1
C9: works_in(a, f(a)) from C6 and C2
C10: manager(g(f(a))) from C8 and C3
C11: manages(g(f(a)), f(a)) from C8 and C4
C12: ¬manages(z, f(a)) ∨ reports_to(a, z) from C9 and C5
C13: reports_to(a, g(f(a))) from C12 and C11
C14: ¬manager(g(f(a))) from C7 and C13
C15: empty clause from C14 and C10

Goal variables: x/a and y/g(f(a))

Exercise 2: Dogs and Animal Lovers

- Everyone who owns a dog is an animal lover
- Animal lovers never harm any animal
- Every dog is an animal
- Tom owns Rex, and Rex is a dog

Goal: Tom does not harm Rex

1. The theory

Goal G: ¬harms(tom, rex)

#	Axiom
A1	∀x [∃y (dog(y) ∧ owns(x, y)) → animal_lover(x)]
A2	∀x [animal_lover(x) → ∀y (animal(y) → ¬harms(x, y))]
A3	∀x [dog(x) → animal(x)]
A4	dog(rex)
A5	owns(tom, rex)

Ex. 2: Conversion to clauses (1/2)

A1

Eliminate →: ∀x [¬∃y (dog(y) ∧ owns(x, y)) ∨ animal_lover(x)]

Push ¬ inward (the ∃ in the antecedent becomes ∀):

∀x [∀y ¬(dog(y) ∧ owns(x, y)) ∨ animal_lover(x)]

De Morgan and drop ∀:

C1: ¬dog(y) ∨ ¬owns(x, y) ∨ animal_lover(x)

A2

Eliminate →: ∀x [¬animal_lover(x) ∨ ∀y (¬animal(y) ∨ ¬harms(x, y))]

Drop ∀:

C2: ¬animal_lover(x) ∨ ¬animal(y) ∨ ¬harms(x, y)

Ex. 2: Conversion to clauses (2/2)

A3, A4 and A5

Eliminate →: ∀x [¬dog(x) ∨ animal(x)]

C3: ¬dog(x) ∨ animal(x)

C4: dog(rex)

C5: owns(tom, rex)

¬G

¬¬harms(tom, rex)

Double negation:

C6: harms(tom, rex)

Ex. 2: Clause set and resolution

Clause set

C1: ¬dog(y) ∨ ¬owns(x, y) ∨ animal_lover(x)
C2: ¬animal_lover(x) ∨ ¬animal(y) ∨ ¬harms(x, y)
C3: ¬dog(x) ∨ animal(x)
C4: dog(rex)
C5: owns(tom, rex)
C6: harms(tom, rex)

Resolution

C7: animal(rex) from C4 and C3
C8: ¬dog(rex) ∨ animal_lover(tom) from C5 and C1
C9: animal_lover(tom) from C8 and C4
C10: ¬animal(rex) ∨ ¬harms(tom, rex) from C9 and C2
C11: ¬harms(tom, rex) from C10 and C7
C12: empty clause from C11 and C6

Substitutions: x/tom and y/rex (C1 and C2), x/rex (C3)

Exercise 3: Paths in a Graph

- If there is an edge from x to y, there is a path from x to y
- If there is an edge from x to y and a path from y to z, there is a path from x to z
- The graph has the edges a→b, b→c and c→d

Goal: There is a path from a to d

1. The theory

Goal G: path(a, d)

#	Axiom
A1	∀x ∀y [edge(x, y) → path(x, y)]
A2	∀x ∀y ∀z [(edge(x, y) ∧ path(y, z)) → path(x, z)]
A3	edge(a, b) ∧ edge(b, c) ∧ edge(c, d)

Ex. 3: Conversion to clauses

A1

Eliminate →: ∀x ∀y [¬edge(x, y) ∨ path(x, y)]

C1: ¬edge(x, y) ∨ path(x, y)

A2

Eliminate →: ∀x ∀y ∀z [¬(edge(x, y) ∧ path(y, z)) ∨ path(x, z)]

De Morgan:

C2: ¬edge(x, y) ∨ ¬path(y, z) ∨ path(x, z)

A3 and ¬G

C3: edge(a, b)

C4: edge(b, c)

C5: edge(c, d)

C6: ¬path(a, d)

No Skolemization is needed: the theory contains no existential quantifier.

Ex. 3: Clause set and resolution

Clause set

C1: ¬edge(x, y) ∨ path(x, y)
C2: ¬edge(x, y) ∨ ¬path(y, z) ∨ path(x, z)
C3: edge(a, b)
C4: edge(b, c)
C5: edge(c, d)
C6: ¬path(a, d)

Resolution

C7: ¬edge(a, y) ∨ ¬path(y, d) from C6 and C2
C8: ¬path(b, d) from C7 and C3
C9: ¬edge(b, y) ∨ ¬path(y, d) from C8 and C2
C10: ¬path(c, d) from C9 and C4
C11: ¬edge(c, d) from C10 and C1
C12: empty clause from C11 and C5

Exercise 4: Advisors and Courses

Every student has a professor who is his/her advisor
Every professor has a course to teach
Every professor that teaches a course is busy
If a professor advises a student, he/she knows the student

Goal: Every student has a busy advisor

1. The theory

#	Axiom
A1	$\forall x [\text{student}(x) \rightarrow \exists y (\text{professor}(y) \wedge \text{advisor}(y, x))]$
A2	$\forall x [\text{professor}(x) \rightarrow \exists y (\text{course}(y) \wedge \text{teaches}(x, y))]$
A3	$\forall x \forall y [(\text{teaches}(x, y) \wedge \text{course}(y)) \rightarrow \text{busy}(x)]$
A4	$\forall x \forall y [\text{advisor}(x, y) \rightarrow \text{knows}(x, y)]$

Goal G: $\forall x [\text{student}(x) \rightarrow \exists y (\text{advisor}(y, x) \wedge \text{busy}(y))]$

A1
Eliminate →: ∀x [¬student(x) ∨ ∃y (professor(y) ∧ advisor(y, x))]
Skolemize: y is in the scope of ∀x, so replace it with a **function** f(x):
∀x [¬student(x) ∨ (professor(f(x)) ∧ advisor(f(x), x))]

Distribute ∨ over ∧ and drop ∀:
C1: ¬student(x) ∨ professor(f(x))
C2: ¬student(x) ∨ advisor(f(x), x)

A2
Eliminate →: ∀x [¬professor(x) ∨ ∃y (course(y) ∧ teaches(x, y))]
Skolemize with g(x):
∀x [¬professor(x) ∨ (course(g(x)) ∧ teaches(x, g(x)))]

Distribute ∨ over ∧ and drop ∀:
C3: ¬professor(x) ∨ course(g(x))
C4: ¬professor(x) ∨ teaches(x, g(x))

A3
Eliminate →: ∀x∀y [¬(teaches(x, y) ∧ course(y)) ∨ busy(x)]
De Morgan:
C5: ¬teaches(x, y) ∨ ¬course(y) ∨ busy(x)

A4
C6: ¬advisor(x, y) ∨ knows(x, y)

¬G
¬∀x [student(x) → ∃y (advisor(y, x) ∧ busy(y))]
Eliminate →: ¬∀x [¬student(x) ∨ ∃y (advisor(y, x) ∧ busy(y))]
Push ¬ inward: ∃x [student(x) ∧ ∀y ¬(advisor(y, x) ∧ busy(y))]
De Morgan: ∃x [student(x) ∧ ∀y (¬advisor(y, x) ∨ ¬busy(y))]
Skolemize: the ∃x is outermost, so it becomes a **constant** *a*:
student(a) ∧ ∀y (¬advisor(y, a) ∨ ¬busy(y))
Split:
C7: student(a)
C8: ¬advisor(y, a) ∨ ¬busy(y)

Clause set
C1: ¬student(x) ∨ professor(f(x))
C2: ¬student(x) ∨ advisor(f(x), x)
C3: ¬professor(x) ∨ course(g(x))
C4: ¬professor(x) ∨ teaches(x, g(x))
C5: ¬teaches(x, y) ∨ ¬course(y) ∨ busy(x)
C6: ¬advisor(x, y) ∨ knows(x, y)
C7: student(a)
C8: ¬advisor(y, a) ∨ ¬busy(y)

Clause set

C1: ¬student(x) ∨ professor(f(x))
C2: ¬student(x) ∨ advisor(f(x), x)
C3: ¬professor(x) ∨ course(g(x))
C4: ¬professor(x) ∨ teaches(x, g(x))
C5: ¬teaches(x, y) ∨ ¬course(y) ∨ busy(x)
C6: ¬advisor(x, y) ∨ knows(x, y)
C7: student(a)
C8: ¬advisor(y, a) ∨ ¬busy(y)

Resolution

C9: professor(f(a)) from C7 and C1
C10: advisor(f(a),a) from C7 and C2
C11: course(g(f(a))) from C3 and C9
C12: ¬ busy(f(a)) from C8 and C10
C13: teaches(f(a),g(f(a))) from C9 and C4
C14: ¬course(g(f(a))) ∨ busy(f(a)) from C5 and C13
C15: busy(f(a)) from C14 and C11
C16: empty clause from C15 and C12

Goal variables: x/a and y/f(a)